数据结构与算法 A

2023 年期中 · 参考答案

依据来源截图并重新核算。第 I.8 题采用截图手写校正后的数组；第 I.11 题来源“树不存在”的判定不成立，本版保留题干并给出数学上成立的答案 153。第三大题来源缺失。

一、选择填空

题号	答案
1	$\Theta(3^n)$、$\Theta(n\sqrt{n})$、$\Theta(2^n)$；从高到低 1、3、2
2	C、D
3	A
4	B、D
5	$\frac{n(n+1)}{2}-1$
6	3 次
7	见下方；是
8	{70,18,10,19,59,36,43,65,28}
9	B
10	4 或 7
11	153（来源答案有误）
12	A、B

1．递推式满足 $T_1(n)+1=3(T_1(n-1)+1)$，得 $T_1(n)=2\times 3^{n-1}-1$。第二式主项 $n^{1.5}$；第三式中 $\left(\sqrt{3}\right)^n$ 慢于 2^n。

2、3．单链表的中点访问需要沿指针行走，因此不能凭有序性得到对数时间；从头寻找删除位置需 $O(n)$。双向链表可沿 prev 找到头再访问第 k 项，C 没有要求常数时间。第 3 题默认仅有头指针，未维护尾指针和长度；若另存这些信息，B、C 也可为常数时间。

4．B 中 c 后无法越过 b 输出 a；D 中 d、c、e 后无法越过 b 输出 a。A、C 可行。

5、6．（5）字符互异使不同位置的子串也互异，总非空子串 $\frac{n(n+1)}{2}$，减去整串。

（6）最优分割为 `d | ata | b | anana`，三刀。可用回文分割递推核验最优性：设 D[i] 为前 i 个字符的最少回文段数，D[0]=0，对所有回文后缀 S[j..i-1] 取 D[i]=min(D[j]+1)。本串 D[1..10]=[1,2,3,2,3,4,5,4,5,4]，故至少 4 段、3 刀。

7．扩充前序为：

```
3,5,6,Φ,Φ,2,7,Φ,Φ,4,Φ,Φ,1,0,Φ,Φ,8,Φ,Φ
```

可以唯一恢复。递归读取：Φ 表示空子树，否则为根，再依次读取左、右子树，边界唯一。

8．对 0 起始下标 3、2、1 依次下沉：

```
处理3：70 65 43 18 59 36 10 19 28
处理2：70 65 10 18 59 36 43 19 28
处理1：70 18 10 19 59 36 43 65 28
```

题目只要求处理 3 个分支结点，根 70 尚未调整。来源印刷答案中把 19、65 写反，手写批注已修正。

10．叶子所在深度至少为 2，因为深度 1 最多 3 个叶子；至多为 3，因为深度 4 至少 16 个叶子。

深度 2：根须有 3 个孩子，每个再有 3 个叶子，内部结点为 1+3=4。

深度 3：根须有 2 个孩子（若有 3 个，至少 12 叶）。第二层内部结点数至少 4；若达到 5 个则至少 10 叶，故恰有 4 个。这 4 个结点的孩子数可为 2、2、2、3，叶子数为 9。内部结点为 1+2+4=7。

11．度数之和即边数：$E = 20 \times 4 + 20 \times 3 + 1 \times 2 + 10 \times 1 = 152$，故 $N = E + 1 = 153$。已知非叶结点 51 个，叶子数 102 个。

> 来源附带答案称“不存在”，并称 153 仅得 1 分；但题干没有限制叶子数，不能据此判不存在。确可构造：把 51 个非叶结点连成一条主链，每个结点剩余孩子位置全部接叶子，叶子总数正好 102。本版将其作为来源错误明确校正。

12．左孩子—右兄弟转换中，两步可为左—右，使 u 成为 v 的父结点；也可为右—右，使 u 与 v 为兄弟。u 的父结点与 v 的父结点为兄弟时，至少还需一次向下转换，不能由上述两步得到。

二、辨析与简答

1．用两个栈 data、mins，同步压入；mins 对每个深度记录截至该深度的最小值。

```
push(x):
    data.push(x)
    if mins.empty(): mins.push(x)
    else: mins.push(min(x, mins.top()))
pop():
    data.pop()
    mins.pop()
get_min(): return mins.top()
```

重复最小值也逐项记录，不会误删。三种操作均为 $O(1)$，辅助空间 $O(n)$。

2．p=abaabc：普通 next=[-1,0,0,1,1,2]，优化 nextval=[-1,0,-1,1,0,2]。

```
下标  012345678901
文本  abaabaabaabc
模式  abaabc          i=5,j=5 处 a≠c，j←2
          abaabc       i=8,j=5 处 a≠c，j←2
             abaabc    全部匹配
起点  0 → 3 → 6
```

匹配起始下标为 6。每次失败，文本指针保持原位，模式指针按 nextval 跳转。

3．第 i 个元素插入堆时，最坏上浮 $O(\log i)$ 层，总计 $\Theta(n \log n)$。不是最优。Floyd 自底向上的建堆让每个分支结点执行 SiftDown，总成本 $\sum_{h \geq 1} O\left(n \frac{h}{2^{h+1}}\right) = O(n)$，这是线性规模输入下的最优渐近界。

4．必要性：u 是 v 的真祖先，深度优先遍历必先进入 u 再进入 v，因此先序 u 在前；完成 v 所在子树之后才退出 u，因此后序 u 在后。

充分性：假设两序关系成立而 u 不是 v 的祖先。v 若是 u 的祖先，则先序 v 在前，矛盾。否则两者分属其最近公共祖先的不同孩子子树。整棵孩子子树被连续访问，先序和后序对这两棵子树的先后次序一致；于是 u 若在先序早于 v，也应在后序早于 v，与条件矛盾。故 u 必为 v 的真祖先。

5．对有序森林的结点总数归纳。设森林第一棵树根为 r，r 的孩子森林为 C，其余兄弟树形成森林 R。

森林后根序列为 `post(C), r, post(R)`。

左孩子—右兄弟二叉树中，r 的左子树对应 C、右子树对应 R，二叉树中序序列为 `in(C), r, in(R)`。

空森林成立；由归纳假设，C、R 上的两种序列分别相等，拼接后亦相等。单棵树是 R 为空的特例，命题成立。

三、来源缺失

给定目录无题目，因此无对应答案。

四、算法设计与实现

1．维护高度严格递增的下标栈，并在末尾加入高度 0 的哨兵。遇到不高于栈顶的柱子，弹出栈顶 j；其最右边界为 i-1，最左边界为弹出后的新栈顶加一。

```
stack = empty
best = 0
for i = 0..n:
    current = (0 if i == n else heights[i])
    while stack is not empty and heights[stack.top()] >= current:
        j = stack.pop()
        left = -1 if stack is empty else stack.top()
        best = max(best, heights[j] * (i-left-1))
    if i < n: stack.push(i)
return best
```

每个柱子至多入栈、出栈一次，时间 $O(n)$，空间 $O(n)$。每个最优矩形可取其最低柱为高度；弹出该高度时会计算其可扩展区间，故答案不会遗漏。相等高度可能提前弹出，但后续同高柱会覆盖更宽区间。样例答案为 10。

2．三叉链表：沿 parent 分别计算 p、q 深度，让较深结点先上行至同深，再同步上行直至相等。

```
LCA_parent(p, q):
    dp = depth_by_parent(p)
    dq = depth_by_parent(q)
    while dp > dq: p = p.parent; dp -= 1
    while dq > dp: q = q.parent; dq -= 1
    while p != q:
        p = p.parent
        q = q.parent
    return p
```

同深后，两结点到最近公共祖先距离相同，第一次相等处即所求。时间 $O(h)$，额外空间 $O(1)$。
二叉链表：已知根，且题设两结点都在树内。后序递归查找：

```
LCA(root, p, q):
    if root is null or root == p or root == q:
        return root
    left = LCA(root.left, p, q)
    right = LCA(root.right, p, q)
    if left is not null and right is not null:
        return root
    return left if left is not null else right
```

两结点在不同子树时根为 LCA；同属一侧则该侧答案上返；一结点是另一结点祖先时，提前返回该结点正合定义。时间 $O(n)$，递归栈 $O(h)$。来源答案采用两条根路径也正确，但需要正确回溯并在找到目标后终止。